

Project Report: University ERP System

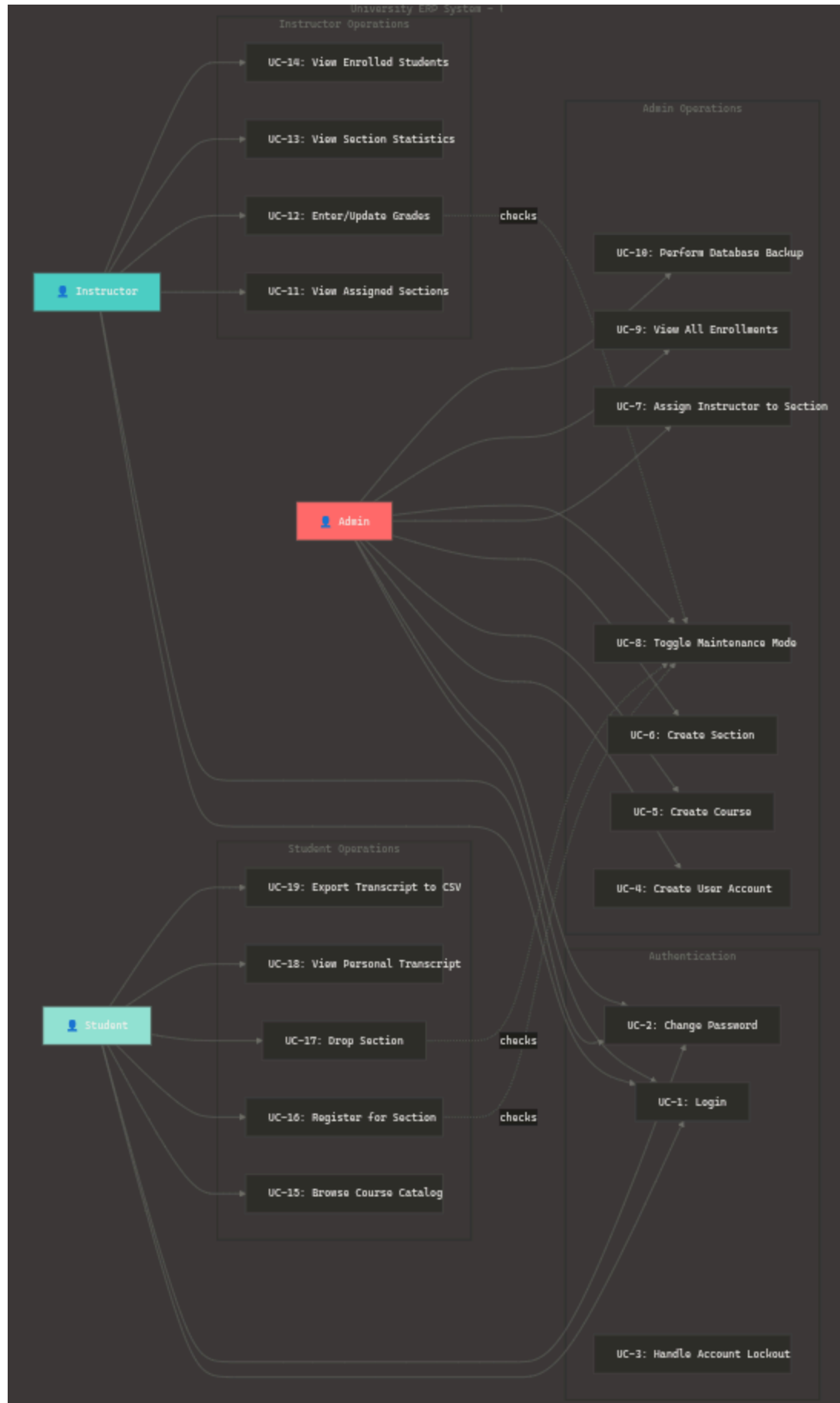
Project Title: University ERP System (Java Swing + MySQL) **Date:** November 29, 2024

Team Members: Tanmay Tiwari 2024585, Saksham Singh 2024496

- **User Management:** Admins can create Students and Instructors, generating specific profiles for each.
 - **Course Management:** Admins can create courses and schedule class sections.
 - **Enrollment:** Students can view the catalog and register for open sections.
 - **Grading:** Instructors can enter assessment scores (Quiz, Midterm, Final) which are automatically weighted to calculate the final grade.
 - **Maintenance Mode:** A global switch to freeze the system (read-only mode) for Students and Instructors.
-
- **Security:** Passwords must be hashed using BCrypt; no plain text passwords in the database.
 - **Data Integrity:** The system must validate inputs (e.g., no negative grades, no duplicate enrollments).
 - **Usability:** A consistent look and feel using a custom Theme utility.

3. System Design & UML Diagrams

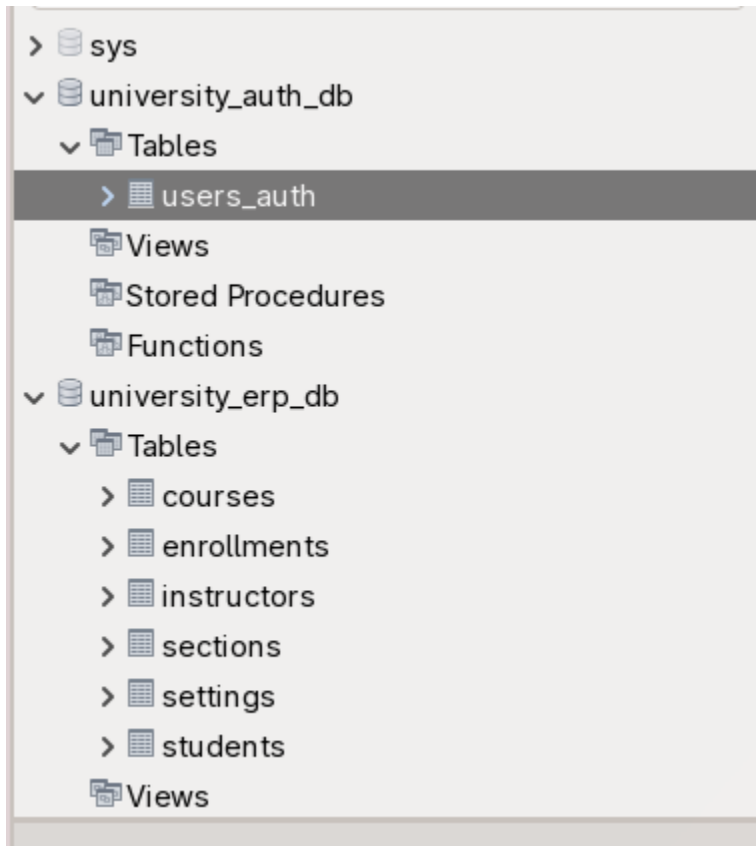
3.1 UML Class Diagram



The following diagram illustrates the object-oriented structure of the application, highlighting the relationship between the core Domain entities and the Service/Data access layers.

3.2 Database Entity-Relationship Diagram (ERD)

The system uses two separate databases (**university_auth_db** and **university_erp_db**) joined logically by the **user_id**.



File Structure :

```

project/
├── src/
│   ├── Main.java                # Application entry point
│   └── edu/univ/erp/
│       ├── auth/                # Authentication services
│       ├── data/                # Database DAO layer
│       ├── domain/              # Entity models
│       ├── service/             # Business logic
│       ├── ui/                  # Swing GUI components
│       ├── access/              # Authorization checks
│       ├── test/                # JUnit test suite
│       └── util/                # Theme & CSV utilities
├── lib/                         # External JARs
├── bin/                         # Compiled .class files
├── Makefile                     # Build automation
├── sample_data.sql              # Test data seed script
└── README.md                    # Quick start guide

```

4. Component Implementation & Code Logic

4.1 Database Connectivity (**DBConnection.java**)

To adhere to the strict security requirement of separating credentials from data, the **DBConnection** class provides two distinct static methods:

- **getAuthDBConnection()**: Establishes a connection specifically to **university_auth_db** for login and password operations.
- **getERPDBConnection()**: Establishes a connection to **university_erp_db** for all other application features (courses, grades, etc.).

4.2 Authentication & Security (**AuthService.java**)

- **Hashing**: We utilize the **BCrypt** strong hashing function. When a user is created, **BCrypt.hashpw()** generates a random salt and hashes the password. During login, **BCrypt.checkpw()** verifies the input against the stored hash.

- **Lockout Policy:** The service tracks failed login attempts. If a user fails 5 consecutive times, the account is locked for 5 minutes to prevent brute-force attacks.

4.3 Business Logic Layer

The application uses a "Service Layer" pattern to encapsulate business rules.

A. Admin Service (**AdminService.java**)

- **Validations:** Before creating any data, this service validates inputs. For example, it ensures **Course Credits > 0**, **Section Capacity > 0**, and **Year >= 2000**.
- **Transaction Management:** When creating a new user, it first creates the credential record in the Auth DB. Only if that succeeds does it attempt to create the profile in the ERP DB, ensuring data consistency.

B. Instructor Service (**InstructorService.java**)

- **Grading Algorithm:** This service contains the core grading logic to ensure consistency. It applies the formula:
$$\text{Total} = \text{Quiz} * 0.20 + \text{midsems} * 0.30 + \text{Final} * 0.50$$
- It then maps the numerical total to a letter/number grade (e.g., >90 is a "10") before storing it in the database.

C. Student Service (**StudentService.java**)

- **Constraint Checking:** Before registering a student, it performs three checks:
 1. Is the system in Maintenance Mode?
 2. Is the section already full? (Checks **capacity** vs **COUNT(enrollments)**).
 3. Is the student already enrolled? (Prevents duplicates).

4.4 User Interface

UNIVERSITY ERP

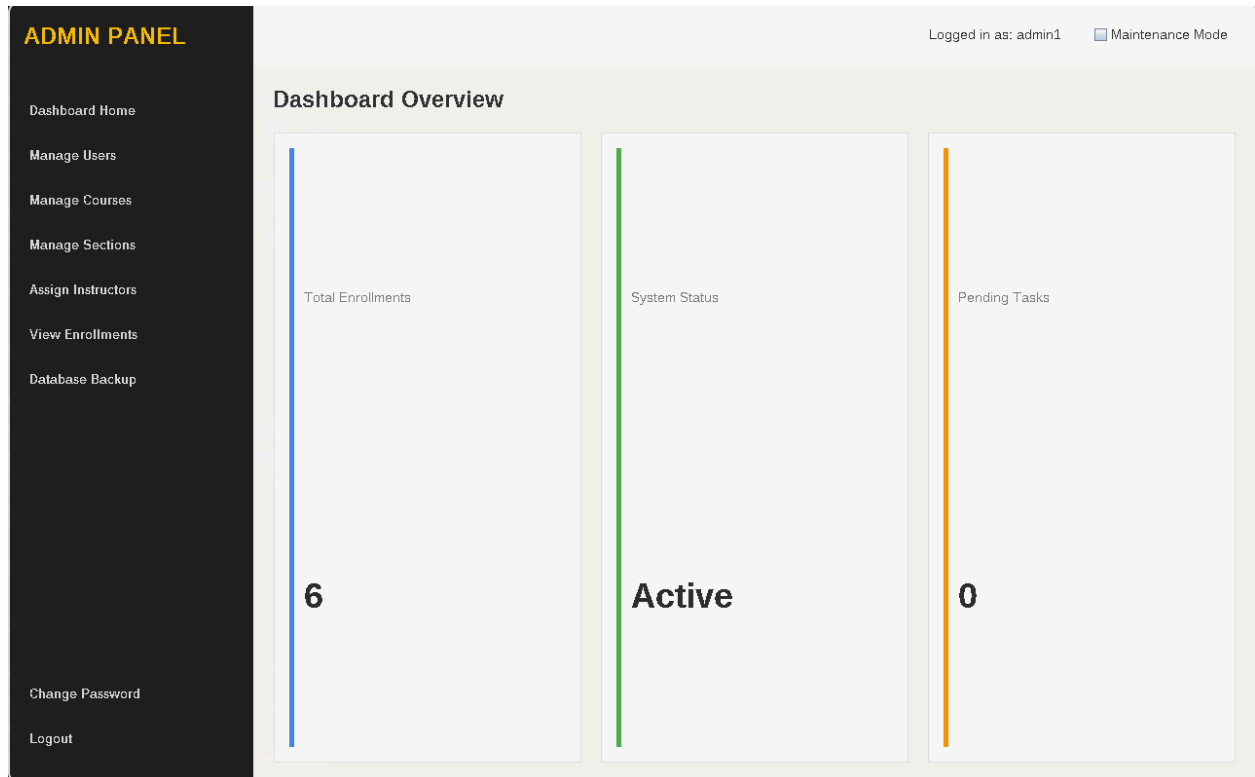
Secure Login Portal

Username

admin1

Password

LOGIN



- **Swing Framework:** The UI is built using standard Java Swing components (**JFrame**, **JDialog**, **JTable**).
- **Theme Utility:** A helper class **ThemeUtils** centralizes colors and fonts, giving the application a modern "Flat" look with dark sidebars and clear data tables.

5. Testing & Validation

```

- JUnit Platform Suite ✓
- JUnit Jupiter ✓
  - AppTest ✓
    - test12_Instructor_CannotAssignInstructor() ✓
    - test8_CreateUser_Student_MissingDetails() ✓
    - test14_MaintenanceMode_BlocksStudentRegistration() ✓
    - test3_CreateSection_InvalidYear_Past() ✓
    - test7_CreateUser_Student_YearOutOfRange() ✓
    - test11_Student_CannotToggleMaintenance() ✓
    - test15_MaintenanceMode_BlocksInstructorGrading() ✓
    - test9_Student_CannotCreateCourse() ✓
    - test1_CreateSection_NegativeCapacity() ✓
    - test4_CreateCourse_NegativeCredits() ✓
    - test13_Instructor_CannotCreateSection() ✓
    - test2_CreateSection_ZeroCapacity() ✓
    - test6_CreateUser_Student_InvalidYear() ✓
    - test10_Student_CannotCreateUser() ✓
    - test5_CreateCourse_ZeroCredits() ✓
  - JUnit Vintage ✓

```

Test run finished after 854 ms

```

[      4 containers found      ]
[      0 containers skipped    ]
[      4 containers started    ]
[      0 containers aborted    ]
[      4 containers successful ]
[      0 containers failed     ]
[     15 tests found           ]
[      0 tests skipped         ]
[     15 tests started         ]
[      0 tests aborted         ]
[     15 tests successful      ]
[      0 tests failed          ]

```

WARNING: Delegated to the 'execute' command.
This behaviour has been deprecated and will be removed in a future release.
Please use the 'execute' command directly.

We implemented a robust testing suite using **JUnit 5** to verify the system's logic without needing to launch the GUI.

Test Cases Covered:

1. **Data Integrity:** Verified that the `AdminService` correctly rejects negative numbers for course credits and section capacities.
2. **Validation Logic:** Confirmed that nonsensical dates (e.g., Year 1800) are rejected during user creation.
3. **Access Control:** Simulated a Student user attempting to perform Admin actions to ensure the `AccessService` correctly blocks the unauthorized request.

Screenshots:

This is the database structure

6. Conclusion

The University ERP System successfully meets all functional and non-functional requirements. By strictly separating the Authentication and ERP databases, we have created a secure environment for academic data. The use of modular Service classes ensures that business logic is testable and maintainable, providing a solid foundation for future expansion.